5 Function Overloading

Chapter Objectives

5.1 Overloading constructor functions

5.2 Creating and using a copy constructor

5.3 The **overload** anachronism

5.4 Using default arguments

5.5 Overloading and ambiguity

5.6 Finding the address of an overloaded function

In this chapter you will learn more about overloading functions. Although this topic was introduced early in this book, there are several further aspects of it that need to be covered. Among the topics included are how to overload constructor functions, how to create a copy constructor, how to give functions default arguments, and how to avoid ambiguity when overloading.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. What is a reference? Give two important uses.
2. Show how to allocate a **float** and an **int** by using **new**. Also, show how to free them by using **delete**.
3. What is the general form of **new** that is used to initialize a dynamic variable? Give a concrete example.
4. Given the following class, show how to initialize a ten-element array so that **x** has the value 1 through 10.

```
class samp
{
     int x;
public:
     samp(int n) { x = n; }
     int getx() { return x; }
};
```

5. Give one advantage of reference parameters. Give one disadvantage.
6. Can dynamically allocated arrays be initialized?
7. Create a function called **mag()** using the following prototype that raises **num** to the order of magnitude specified by **order**:

```
void mag(long &num, long order);
```

   For example, if **num** is 4 and order is 2, when **mag()** returns, **num** will be 400. Demonstrate in a program that the function works.

5.1 OVERLOADING CONSTRUCTOR FUNCTIONS

It is possible-indeed, common-to overload a class's constructor function. (It is not possible to overload a destructor, however.) There are three main reasons why you will want to overload a constructor function: to gain flexibility, to support arrays, and to create copy constructors. The first two of these are discussed in this section. Copy constructors are discussed in the next section.

One thing to keep in mind as you study the examples is that there must be a constructor function for each way that an object of a class will be created. If a program attempts to create an object for which no matching constructor is found, a compile-time error occurs. This is why overloaded constructor functions are so common to C++ programs.

EXAMPLES

1. Perhaps the most frequent use of overloaded constructor functions is to provide the option of either giving an object an initialization or not giving it one. For example, int the following program, **o1** is given an initial value, but **o2** is not. If you remove thee constructor that has the empty argument list, the program will not compile because there is no constructor that matches a non-initialized object of type **myclass**. The reverse is also true. If you remove the parameterized constructor, the program will not compile because there is no match for an initialized object. Both are needed for this program to compile correctly.

```
#include <iostream>
using namespace std;

class myclass
{
    int x;
public:
    // overload constructor two ways
    myclass() { x = 0; } // no initializer
    myclass(int n) { x = n; } // initializer
    int getx() { return x; }
};

int main()
{
    myclass o1(10); // declare with initial value
    myclass o2; // declare without initializer

    cout << "o1: " << o1.getx() << '\n';
    cout << "o2: " << o2.getx() << '\n';

    return 0;
}
```

2. Another common reason constructor functions are overloaded is to allow both individual objects and arrays of objects to occur within a program. As you probably know from your own programming experience, it is fairly common to initialize a single variable, but it is not as common to initialize an array. (Quite often array values are assigned using information known only when the program is executing.) Thus, to allow non-initialized arrays of objects along with initialized objects, you must include a constructor that supports initialization and one that does not.

   For instance, assuming the class **myclass** from Example 1, both of these declarations are valid:

```
myclass ob(10);
myclass ob[5];
```

   By providing both a parameterized and a parameterless constructor, your program allows the creation of objects that are either initialized or not as needed.

   Of course, once you have defined both parameterized and parameterless constructors you can use them to create initialized and non-initialized arrays. For example, the following program declares two arrays of type **myclass**; one is initialized and the other is not:

```
#include <iostream>
using namespace std;

class myclass
{
    int x;
public:
    // overload constructor two ways
    myclass() { x = 0; } // no initializer
    myclass(int n) { x = n; } // initializer
    int getx() { return x; }
};

int main()
{
    myclass o1[10]; // declare array without initializers

    // declare without initializer
    myclass o2[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int i;

    for(i=0; i<10; i++)
    {
        cout << "o1[" << i << "]: " << o1[i].getx();
        cout << '\n';
        cout << "o2[" << i << "]: " << o2[i].getx();
        cout << '\n';
    }

    return 0;
}
```

In this example, all elements of **o1** are set to 0 by the constructor function. The elements of **o2** are initialized as shown in the program.

3. Another reason for overloading constructor functions is to allow the programmer to select the most convenient method of initializing an object. To see how, first examine the next example, which creates a class that holds a calender date. It overloads the **date()** constructor two ways. One form accepts the date as a character string. In the other form, the date is passed as three integers.

```
#include <iostream>
#include <cstdio> // included for sscanf()
using namespace std;

class date
{
    int day, month, year;
public:
    date(char *str);
    date(int m, int d, int y)
    {
```

```
        day = d;
        month = m;
        year = y;
    }
    void show()
    {
        cout << month << '/' << day << '/';
        cout << year << '\n';
    }
};

date::date(char *str)
{
    sscanf(str, "%d%*c%d%*c%d", &month, &day, &year);
}

int main()
{
    // construct date object using string
    date sdate("12/31/99");

    // construct date object using integers
    date idate(12, 31, 99);

    sdate.show();
    idate.show();

    return 0;
}
```

The advantage of overloading the **date()** constructor, as shown in this program, is that you are free to use whichever version most conveniently fits the situation in which it is being used. For example, if a **date** object is being created from user input, the string version is the easiest to use. However, if the **date** object is being constructed through some sort of internal computation, the three-integer parameter version probably makes more sense.

Although it is possible to overload a constructor as many times as you want, doing so excessively has a destructuring effect on the class. From a stylistic point of view, it is best to overload a constructor to accommodate only those situations that are likely to occur frequently. For example, overloading **date()** a third time so the date can be entered in terms of milliseconds makes little sense. However, overloading it to accept an object of type **time_t** (a type that stores the system date and time) could be very valuable. (See the Mastery Skills Check exercises at the end of this chapter for an example that does just this.)

4. There is one other situation in which you will need to overload a class's constructor function: when a dynamic array of that class will be allocated. As you should recall from the preceding chapter, a dynamic array cannot be initialized. Thus, if the class contains a constructor that takes an initializer, you must include an overloaded version that takes no initializer. For example, here is a program that allocates an object array dynamically:

```
#include <iostream>
```

```
using namespace std;

class myclass
{
    int x;
public:
    // overload constructor two ways
    myclass() { x = 0; } // no initializer
    myclass(int n) { x = n; } // initializer
    int getx() { return x; }
};

int main()
{
   myclass *p;
   myclass ob(10); // initialize single variable

    p = new myclass[10]; // can't use initializers here
    if(!p)
    {
        cout << "Allocation error\n";
        return 1;
    }

    int i;

    // initialize all elements to ob
    for(i=0; i<10; i++)
        p[i] = ob;

    for(i=0; i<10; i++)
    {
        cout << "p[" << i << "]: " << p[i].getx();
        cout << '\n';
    }

    return 0;
}
```

Without the overloaded version of **myclass()** that has no initializer, the **new** statement would have generated a compile-time error and the program would not have been compiled.

EXERCISES

1. Given this partially defined class

```
class strtype
{
    char *p;
    int len;
public:
    char *getstring { return p; }
```

```
    int getlength() { return len; }
};
```

add two constructor functions. Have the first one take no parameters. Have this one allocate 255 bytes of memory (using **new**), initialize that memory as a null string, and give **len** a value of 255. Have the other constructor take two parameters. The first is the string to use for initialization and the other is the number of bytes to allocate. Have this version allocate the specified amount of memory and copy the string ti that memory. Perform all necessary boundary checks and demonstrate that your constructors work by including a short program.

2. In Exercise 2 of Chapter 2, Section 2.1, you created a stopwatch emulation. Expand your solution so that the **stopwatch** class provides both a parameterless constructor (as it does already) and an overloaded version that accepts the system time in the form returned by the standard function **clock()**. Demonstrate that your improvement works.

3. On your own, think about ways in which an overloaded constructor function can be beneficial to your own programming tasks.

5.2 CREATING AND USING A COPY CONSTRUCTOR

One of the more important forms of an overloaded constructor is the *copy constructor*. As numerous examples from the preceding chapters have shown, problems can occur when an object is passed to or returned from a function. As you will learn in this section, one way to avoid these problems is to define a copy constructor.

To begin, let's restate the problem that a copy constructor is designed to solve. When an object is passed to a function, a bitwise (i.e., exact) copy of that object is made and given to the function parameter that receives the object. However, there are cases in which this identical copy is not desirable. For example, if the object contains a pointer to allocated memory, the copy will point to the *same* memory as does the original object. Therefore, if the copy makes a change to the contents of this memory, it will be changed for the original object too! Also, when the function terminates, the copy will be destroyed, causing its destructor to be called. This might lead to undesired side effects that further affect the original object.

A similar situation occurs when an object is returned by a function. The compiler will commonly generate a temporary object that holds copy of the value returned by the function. (This is done automatically and is beyond your control.) This temporary object goes out of scope once the value is returned to the calling routine, causing the temporary object's destructor to be called. However, if the destructor destroys something needed by the calling routine (for example, if it frees dynamically allocated memory), trouble will follow.

At the core of these problems is the fact that a bitwise copy of the object is being made. To prevent these problems, you, the programmer, need to define precisely what occurs when a copy of an object is made so that you can avoid undesired side effects. The way you accomplish this is by creating a copy constructor. By defining a copy constructor, you can fully specify exactly what occurs when a copy of an object is made.

It is important for you to understand that C++ defines two distinct types of situations in which the value of one object is given to another. The first situation is assignment. The second situation is initialization which can occur three ways:

- when an object is used to initialize another in a declaration statement.
- when an object is passed as a parameter to a function, and
- when a temporary object is created for use as a return value of a function.

The copy constructor only applies to initializations. It does not apply to assignments.
By default, when an initialization occurs, the compiler will automatically provide a bitwise copy. (That is, C++ automatically provides a default copy constructor that simply duplicates the object.) However, it is possible to specify precisely how one object will initialize another by defining a copy constructor. Once defined, the copy constructor is called whenever an object is used to initialize another.
Remember: *Copy constructors do not affect assignment operations.*
The most common form of copy constructor is shown here:

```
classname(const classname &obj)
{
    // body of constructor
}
```

Here *obj* is a reference to an object that is being used to initialize another object. For example, assuming a class called **myclass**, and that **y** is an object of type **myclass**, the following statements would invoke the **myclass** copy constructor:

```
myclass x = y; // y explicitly initializing x
fun1(y);       // y passed as a parameter
y = func2();   // y receiving a returned object
```

In the first two cases, a reference to **y** would be passed to the copy constructor. In the third, a reference to the object returned by **func()** is passed to the copy constructor.

EXAMPLES

1. Here is an example that illustrates why an explicit copy constructor function is needed. This program creates a very limited "safe" integer array type that prevents array boundaries from being overrun. Storage for each array is allocated using **new**, and a pointer to the memory is maintained within each array object.

```
/*
   This program creates a "safe" array class. Since space
   for the array is dynamically allocated, a copy constructor
    is provided to allocate memory when one array object is
    used to initialize another.
*/
#include <iostream>
#include <cstdlib>
using namespace std;

class array
{
    int *p;
    int size;
public:
    array(int sz) // constructor
    {
        p = new int[sz];
        if(!p)
            exit(1);
        size = sz;
        cout << "Using 'normal' constructor\n";
```

```
    }
    ~array() { delete [] p; }

    //copy constructor
    array(const array &a);

    void put(int i, int j)
    {
        if(i>=0 && i<size)
            p[i] = j;
    }
    int get(int i)
    {
        return p[i];
    }
};

/*
   Copy constructor

   In the following, memory is allocated specifically
   for the copy, and the address of this memory is assigned
    to p. Therefore, p is not pointing to the same
    dynamically allocated memory as the original object.
*/
array::array(const array &a)
{
    int i;

    size = a.size;
    p = new int[a.size]; // allocate memory for copy
    if(!p)
        exit(1);
    for(i=0; i<a.size; i++)
        p[i] = a.p[i]; // copy contents
    cout << "Using copy constructor\n";
}

int main()
{
    array num(10); // this calls "normal" constructor
    int i;

    // put some values into the array
    for(i=0; i<10; i++)
        num.put(i, i);

    // display num
    for(i=9; i>=0; i--)
        cout << num.get(i);
    cout << "\n";
```

```
    // create another array and initialize with num
    array x = num; // this invokes copy constructor

    // display x
    for(i=0; i<10; i++)
        cout << x.get(i);

    return 0;
}
```

When **num** is used to initialize **x**, the copy constructor is called, memory for the new array is allocated and stored in **x.p**, and the contents of **num** are copied to **x**'s array. In this way, **x** and **num** have arrays that have the same values, but each array is separate and distinct. (That is, **num.p** and **x.p** do not point to the same piece of memory.) If the copy constructor had not been created, the bitwise initialization **array x = num** would have resulted in **x** and **num** sharing the same memory for their arrays! (That is, **num.p** and **x.p** would have, indeed, pointed to the same location.)

The copy constructor is called only for initializations. For example, the following sequence does not call the copy constructor defined in the preceding program:

```
array a(10);
array b(10);

b = a; // does not call copy constructor
```

In this case, **b = a** performs the assignment operation.

2. To see how the copy constructor helps prevent some of the problems associated with passing certain types of objects to functions, consider this (incorrect) program:

```
// This program has an error.
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
public:
    strtype(char *s);
    ~strtype() { delete [] p; }
    char *get() { return p; }
};

strtype::strtype(char *s)
{
    int l;

    l = strlen(s)+1;

    p = new char [l];
```

```
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    strcpy(p, s);
}

void show(strtype x)
{
    char *s;

    s = x.get();
    cout << s << "\n";
}

int main()
{
    strtype a("Hello"), b("There");

    show(a);
    show(b);

    return 0;
}
```

In this program, when a **strtype** object is passed to **show()**, a bitwise copy is made (since no copy constructor has been defined) and put into parameter **x**. Thus, when the function returns, **x** goes out of scope and is destroyed. This, of course, causes **x**'s destructor to be called, which frees **x.p**. However, the memory being freed is the same memory that is still being used by the object used to call the function. This results in an error.

The solution to the preceding problem is to define a copy constructor for the **strtype** class that allocates memory for the copy when the copy is created. This approach is used by the following, corrected, program:

```
/*
   This program uses a copy constructor to allow strtype
      objects
   to be passed to functions.
*/
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
public:
    strtype(char *s); // constructor
    strtype(const strtype &o); // copy constructor
```

```
    ~strtype() { delete [] p; } // destructor
    char *get() { return p; }
};

// "Normal" constructor
strtype::strtype(char *s)
{
    int l;

    l = strlen(s)+1;

    p = new char [l];
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    strcpy(p, s);
}

// Copy constructor
strtype::strtype(const strtype &o)
{
    int l;

    l = strlen(o.p)+1;

    p = new char [l]; // allocate memory for new copy
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    strcpy(p, o.p); // copy string into copy
}

void show(strtype x)
{
    char *s;

    s = x.get();
    cout << s << "\n";
}

int main()
{
    strtype a("Hello"), b("There");

    show(a);
```

```
    show(b);

    return 0;
}
```

Now when **show()** terminates and **x** goes out of scope, the memory pointed to by **x.p** (which will be freed) is not the same as the memory still in use by the object passed to the function.

EXERCISES

1. The copy constructor is also invoked when a function generates the temporary object that is used as the function's return value (for those functions that return objects). With this in mind, consider the following output:

```
Constructing normally
```

```
Constructing normally
```

```
Constructing copy
```

This output was created by the following program. Explain why, and describe precisely what is occurring.

```
#include <iostream>
using namespace std;

class myclass
{
public:
    myclass();
    myclass(const myclass &o);
    myclass f();
};

// Normal constructor
myclass::myclass()
{
    cout << "Constructing normally\n";
}

// Copy constructor
myclass::myclass(const myclass &o)
{
    cout << "Constructing copy\n";
}

// Return an object.
myclass myclass::f()
{
    myclass temp;

    return temp;
}
```

```
int main()
{
    myclass obj;

    obj = obj.f();

    return 0;
}
```

2. Explain what is wrong with the following program and then fix it.

```
// This program contains an error.
#include <iostream>
#include <cstdlib>
using namespace std;

class myclass
{
    int *p;
public:
    myclass(int i);
    ~myclass() { delete p; }
    friend int getval(myclass o);
};

myclass::myclass(int i)
{
    p = new int;

    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    *p = i;
}

int getval(myclass o)
{
    return *o.p; // get value
}

int main()
{
    myclass a(1), b(2);

    cout << getval(a) << " " << getval(b);
    cout << "\n";
    cout << getval(a) << " " << getval(b);

    return 0;
}
```

3. In your own words, explain the purpose of a copy constructor and how it differs from a normal constructor.

5.3 THE OVERLOAD ANACHRONISM

When C++ was first invented, the keyword **overload** was required to create an overloaded function. Although **overload** is now obsolete and no longer supported by modern C++ compilers, you may still see **overload** used in old programs, so it is a good idea to understand how it was applied.
The general form of **overload** is shown here,

```
overload func_name;
```

where *func-name* is the name of the function to be overloaded. This statement must precede the overloaded function declarations. For example, this tells the compiler that you will be overloading a function called **time()**:

```
overload timer;
```

Remember: ***overload*** is obsolete and no longer supported by modern C++ compilers.

5.4 USING DEFAULT ARGUMENTS

There is a feature of C++ that is related to function overloading. This feature is called the *default argument*, and it allows you to give a parameter a default value when no corresponding argument is specified when the function is called. As you will see, using default arguments is essentially a shorthand form of function overloading.
To give a parameter a default argument, simply follow that parameter with an equal sign and the value you want it to default to if no corresponding argument is present when the function is called. For example, this function gives its two parameters default values of 0:

```
void f(int a=0, int b=0);
```

Notice that this syntax is similar to variable initialization. This function can now be called three different ways. First, it can be called with both arguments specified. Second, it can be called with only the first argument specified. In this case, **b** will default to 0. Finally, **f()** can be called with no arguments, causing both **a** and **b** to default to 0. That is the following invocations of **f()** are all valid:

```
f(); // a and b default to 0
f(10); // a is 10, b defaults to 0
f(10, 99); // a is 10, b is 99
```

In this example, it should be clear that there is no way to default **a** and specify **b**.
When you create a function that has one or more default arguments, those arguments must be specified only once: either in the function's prototype or in its definition if the definition precedes the function's first use. The defaults cannot be specified in both the prototype and the definition. This rule applies even if you simply duplicate the same defaults.
As you can probably guess, all default parameters must be to the right of any parameters that don't have defaults. Further, once you begin to define default parameters, you cannot specify any parameters that have no defaults.
One other point about default arguments: they must be constants or global variables. They cannot be local variables or other parameters.

EXAMPLES

1. Here is a program that illustrates the example described in the preceding discussion:

```
// A simple first example of default arguments.
#include <iostream>
using namespace std;

void f(int a=0, int b=0)
{
     cout << "a: " << a << ", b: " << b;
     cout << '\n';
}

int main()
{
     f();
     f(10);
     f(10, 99);

     return 0;
}
```

As you should expect, this program displays the following output:

```
a: 0, b: 0
a: 10, b: 0
a: 10, b: 99
```

Remember that once the first default argument is specified, all following parameters must have defaults as well. For example, this slightly different version of **f()** causes a compile-time error:

```
void f(int a=0, int b) // wrong! b must have default, too
{
     cout << "a: " << a << ", b: " << b;
     cout << '\n';
}
```

2. To understand how default arguments are related to function overloading, first consider the next program, which overloads the function called **rect_area()**. This function returns the area of a rectangle.

```
// Compute area of a rectangle using overloaded functions.
#include <iostream>
using namespace std;

// Return area of a non-square rectangle.
double rect_area(double length, double width)
{
     return length * width;
}

// Return area of a square
```

```
double rect_area(double length)
{
    return length * length;
}

int main()
{
    cout << "10 x 5.8 rectangle has area: ";
    cout << rect_area(10.0, 5.8) << '\n';

    cout << "10 x 10 square has area: ";
    cout << rect_area(10.0) << '\n';

    return 0;
}
```

Int his program, **rect_area()** is overloaded two ways. In the first way, both dimensions of a rectangle are passed to the function. This version is used when the rectangle is not a square. However, when the rectangle is a square, only one argument need to be specified, and the second version of **rect_area()** is called.

If you think about it, it is clear that in this situation there is really no need to have two different functions. Instead, the second parameter can be defaulted to some value that acts as a flag to **rect_area()**. When this value is seen by the function, it uses the **length** parameter twice. Here is an example of this approach:

```
// Compute area of a rectangle using default arguments.
#include <iostream>
using namespace std;

// Return area of a rectangle.
double rect_area(double length, double width = 0)
{
    if(!width)
        width = length;
    return length * width;
}

int main()
{
    cout << "10 x 5.8 rectangle has area: ";
    cout << rect_area(10.0, 5.8) << '\n';

    cout << "10 x 10 square has area: ";
    cout << rect_area(10.0) << '\n';

    return 0;
}
```

Here 0 is the default value of **width**. This value was chosen because no rectangle will have a width of 0. (Actually, a rectangle with a width of 0 is a line.) Thus, if this default value is seen, **rect_area()** automatically uses the value in **length** for the value of **width**.

As this example shows, default arguments often provide a simple alternative to function

overloading. (Of course, there are many situations in which function overloading is still required.)

3. It is not only legal to give constructor functions default arguments, it is also common. As you saw earlier in this chapter, many times a constructor is overloaded simply to allow both initialized and uninitialized objects to be created. In many cases, you can avoid overloading a constructor by giving it one or more default arguments. For example, examine this program:

```
#include <iostream>
using namespace std;

class myclass
{
    int x;
public:
    /*
       Use default argument instead of overloading
       myclass's constructor.
    */
    myclass(int n = 0) { x = n; }
    int getx() { return x; }
};

int main()
{
    myclass o1(10); // declare with initial value
    myclass o2; // declare without initializer

    cout << "o1: " << o1.getx() << '\n';
    cout << "o2: " << o2.getx() << '\n';

    return 0;
}
```

As this example shows, by giving **n** the default value of 0, it is possible to create objects that have explicit initial values and those for which the default value is sufficient.

4. Another good application for a default argument is found when a parameter is used to select an option. It is possible to give that parameter a default value that is used as a flag that tells the function to continue to use the previously selected option. For example, in the following program, the function **print()** displays a string on the screen. If its **how** parameter is set to **ignore**, the text is displayed as is. If **how** is **upper**, the text is displayed in uppercase. If **how** is **lower**, the text is displayed in lowercase. When **how** is not specified, it defaults to -1, which tells the function to reuse the last **how** value.

```
#include <iostream>
#include <cctype>
using namespace std;

const int ignore = 0;
const int upper = 1;
const int lower = 2;
```

```
void print(char *s, int how = -1);

int main()
{
    print("Hello There\n", ignore);
    print("Hello There\n", upper);
    print("Hello There\n"); // continue in upper
    print("Hello There\n", lower);
    print("That's all\n"); // continue in lower

    return 0;
}

/*
   Print a string in the specified case. Use
   last case specified if none is given.
*/

void print(char *s, int how)
{
    static int oldcase = ignore;

    // reuse old case if none specified
    if(how<0)
        how = oldcase;
    while(*s)
    {
        switch(how)
        {
            case upper: cout << (char) toupper(*s);
            break;
            case lower: cout << (char) tolower(*s);
            break;
            default: cout << *s;
        }
        s++;
    }
    oldcase = how;
}
```

This function displays the following output:

```
Hello There
HELLO THERE
HELLO THERE
hello there
that's all
```

5. Earlier in this chapter, you saw the general form of a copy constructor. This general form was shown with only one parameter. However, it is possible to create copy constructors that take additional arguments, as long as the additional arguments have default values. For example, the following is also an acceptable form of a copy constructor:

```
myclass(const myclass &obj, int x = 0)
{
    // body of constructor
}
```

As long as the first argument is a reference to the object being copied, and all other arguments default, the function qualifies as a copy constructor. This flexibility allows you to create copy constructors that have other uses.

6. Although default arguments are powerful and convenient, when used correctly, default arguments allow a function to perform its job in an efficient and easy-to-use manner. However, this is only the case when the default value given to a parameter makes sense. For example, if the argument is the value wanted nine times out of ten, giving a function a default argument to this effect is obviously a good idea. However, in cases in which no one value is more likely to be used than another, or when there is no benefit to using a default argument as a flag value, it makes little sense to provide a default value. Actually, providing a default argument when one is not called for destructures your program and tends to mislead anyone else who has to use that function.

   As with function overloading, part of becoming an excellent C++ programmer is knowing when to use a default argument and when not to.

EXERCISES

1. In the C++ standard library is the function **strtoi()**, which has this prototype:

   ```
   long strtoi(const char *start, const **end, int base);
   ```

   The function converts the numeric string pointed to by *start* into a long integer. The number base of the numeric string is specified by *base*. Upon return, *end* points to the character in the string immediately following the end of the number. The long integer equivalent of the numeric string is returned. *base* must be in the range 2 to 38. However, most commonly, base 10 is used.

   Create a function called **mystrtoi()** that works the same as **strtoi()** except that *(*base) is given the default argument of 10. (Feel free to use **strtoi()** to actually perform the conversion. It requires the header **<cstdlib>**.) Demonstrate that your version works correctly.

2. What is wrong with the following function prototype?

   ```
   char *f(char *p, int x = 0, char *q);
   ```

3. Most C++ compilers supply nonstandard functions that allow cursor positioning and the like. If your compiler supplies such functions, create a function called **myclreol()** that clears the line from the current cursor position to the end of the line. However, give this function a parameter that specifies the number of character positions to clear. If the parameter is not specified, automatically clear the entire line. Otherwise, clear only the number of character positions specified by the parameter.

4. What is wrong with the following prototype, which uses a default argument?

   ```
   int f(int count, int max = count);
   ```

5.5 OVERLOADING AND AMBIGUITY

When you are overloading functions, it is possible to introduce ambiguity into your program. Overloading-caused ambiguity can be introduced through type conversions, reference parameters, and default arguments. Further, some types of ambiguity are caused by the overloaded functions themselves. Other types occur in the manner in which an overloaded function is called. Ambiguity must be removed before your program will compile without error.

EXAMPLES

1. One of the most common types of ambiguity is caused by C++'s automatic type conversion rules. As you know, when a function is called with an argument that is of a compatible (but not the same) type as the parameter to which it is being passed, the type of the argument is automatically converted to the target type. In fact, it is this sort of type conversion that allows a function such as **putchar()** to be called with a character even though its argument is specified as an **int**. However, in some cases, this automatic type conversion will cause an ambiguous situation when a function is overloaded. To see how, examine this program:

```
// This program contains an ambiguity error.
#include <iostream>
using namespace std;

float f(float i)
{
    return i / 2.0;
}

double f(double i)
{
    return i / 3.0;
}

int main()
{
    float x = 10.09;
    double y = 10.09;

    cout << f(x); // unambiguous - use f(float)
    cout << f(y); // unambiguous - use f(double)

    cout << f(10); // ambiguous, convert 10 to double or
       float??

    return 0;
}
```

As the comments in **main()** indicate, the compiler is able to select the correct version of **f()** when it is called with either a **float** or a **double** variable. However, what happens when it is called with an integer? Does the compiler call **f(float)** or **f(double)**? (Both are valid conversions!) In either case, it is valid to promote an integer into either a **float** or a **double**. Thus, the ambiguous situation is created.

This example also points out that ambiguity can be introduced by the way an overloaded function is called. The fact is that there is no inherent ambiguity in the overloaded versions of **f()** as long as each is called with an unambiguous argument.

2. Here is another example of function overloading that is not ambiguous in and of itself. However, when this function is called with the wrong type of argument, C++'s automatic conversion rules cause an ambiguous situation,

```
// This program is ambiguous.
#include <iostream>
using namespace std;

void f(unsigned char c)
{
     cout << c;
}

void f(char c)
{
     cout << c;
}

int main()
{
     f('c');
     f(86); // which f() is called???

     return 0;
}
```

Here, when **f()** is called with the numeric constant 86, the compiler cannot know whether to call **f(unsigned char)** or **f(char)**. Either conversion is equally valid, thus leading to ambiguity.

3. One type of ambiguity is caused when you try to overload functions in which the only difference is the fact that one uses a reference parameter and the other uses the default call-by-value parameter. Given C++'s formal syntax, there is no way for the compiler to know which function to call. Remember, there is no syntactical difference between calling a function that takes a value parameter and calling a function that takes a reference parameter. For example:

```
// An ambiguous program.
#include <iostream>
using namespace std;

int f(int a, int b)
{
     return a+b;
}

// this is inherently ambiguous
int f(int a, int &b)
{
     return a-b;
```

```
}

int main()
{
    int x=1, y=2;

    cout << f(x, y); // which version of f() is called???

    return 0;
}
```

Here, **f(x, y)** is ambiguous because it could be calling either version of the function. In fact, the compiler will flag an error before this statement is even specified because the overloading of the two functions is inherently ambiguous and no reference to them could be resolved.

4. Another type of ambiguity is caused when you are overloading a function in which one or more overloaded functions use a default argument. Consider this program:

```
// Ambiguity based on default arguments plus overloading.
#include <iostream>
using namespace std;

int f(int a)
{
    return a*a;
}

int f(int a, int b = 0)
{
    return a*b;
}

int main()
{
    cout << f(10, 2); // calls f(int, int)
    cout << f(10); // ambiguous - call f(int) or f(int, int)?

    return 0;
}
```

Here the call **f(10, 2)** is perfectly acceptable, and unambiguous. However, the compiler has now way of knowing whether the call **f(10)** is calling the first version of **f()** or the second version with **b** defaulting.

EXERCISE

1. Try to compile each of the preceding ambiguous programs. Make a mental note of the types of error messages they generate. This will help you recognize ambiguity errors when they creep into your own programs.

5.6 FINDING THE ADDRESS OF AN OVERLOADED FUNCTION

To conclude this, chapter, you will learn how to find the address of an overloaded function. Just as in C, you can assign the address of a function (that is, its entry point) to a pointer and access that function via that pointer. A function's address is obtained by putting its name on the right side of an assignment statement without any parentheses or arguments. For example, if **zap()** is a function, assuming proper declarations, this is a valid way to assign **p** the address of **zap()**:

```
p = zap;
```

In C, any type of pointer can be used to point to a function because there is only one function that it can point to. However, in C++ the situation is a bit more complex because a function can be overloaded. Thus, there must be some mechanism that determines which function's address is obtained.
The solution is both elegant and effective. When obtaining the address of an overloaded function, it is *the way the pointer is declared* that determines which overloaded function's address will be obtained. In essence, the pointer's declaration is matched against those of the overloaded functions. The function whose declaration matches is the one whose address is used.

EXAMPLE

1. Here is a program that contains two versions of a function called **space()**. The first version outputs **count** number of spaces to the screen. The second version outputs **count** number of whatever type of character is passed to **ch**. In **main()**, two function pointers are declared.The first one is specified as a pointer to a function having only one integer parameter. The second is declared as a pointer to a function taking two parameters.

```
/*
   Illustrate assigning function pointers to
   overloaded functions.
*/
#include <iostream>
using namespace std;

// Output count number of spaces.
void space(int count)
{
    for( ; count; count--)
        cout << ' ';
}

// Output count number of chs.
void space(int count, char ch)
{
    for( ; count; count--)
        cout << ch;
}

int main()
{
    /*
```

```
        Create a pointer to void function with
        one int parameter
    */
    void (*fp1)(int);

    /*
        Create a pointer to void function with
        one int parameter and one character parameter.
    */
    void (*fp2)(int, char);

    fp1 = space; // gets address of space(int)

    fp2 = space; // gets address of space(int, char)

    fp1(22); // output 22 spaces
    cout << "|\n";

    fp2(30, 'x'); // output 30 x's
    cout << "|\n";

    return 0;
}
```

As the comments illustrate, the compiler is able to determine which overloaded function to obtain the address of based upon how **fp1** and **fp2** are declared.

To review: When you assign the address of an overloaded function to a function pointer, it is the declaration of the pointer that determines which function's address is assigned. Further, the declaration of the function pointer must exactly match one and only one of the overloaded functions. If it does not, ambiguity will be introduced, causing a compile-time error.

EXERCISE

1. Following are two overloaded functions. Show how to obtain the address of each.

```
int dif(int a, int b)
{
    return a-b;
}

float dif(float a, float b)
{
    return a-b;
}
```

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Overload the **date()** constructor from Section 5.1, Example 3, so that it accepts a parameter of type **time_t**. (Remember, **time_t** is a type defined by the standard time and date functions found in your C++ compiler's library.)

2. What is wrong with the following fragment?

```
class samp
{
    int a;
public:
    samp(int i) { a = i; }
    // ...
};

//...

int main()
{
    samp x, y(10);

    // ...
}
```

3. Give two reasons why you might want (or need) to overload a class's constructor.

4. What is the most common general form of a copy constructor?

5. What type of operations will cause the copy constructor to be invoked?

6. Briefly explain what the **overload** keyword does and why it is no longer needed.

7. Briefly describe a default argument.

8. Create a function called **reverse()** that takes two parameters. The first parameter, called **str**, is a pointer to a string that will be reversed upon return from the function. The second parameter is called **count**, and it specifies how many characters of **str** to reverse. Give **count** a default value that, when present, tells **reverse()** to reverse the entire string.

9. What is wrong with the following prototype?

```
char *wordwrap(char *str, int size = 0, char ch);
```

10. Explain some ways that ambiguity can be introduced when you are overloading functions.

11. What is wrong with the following fragment?

```
void compute(double *num, int divisor=1);
void compute(double *num);
// ...
compute(&x);
```

12. When you are assigning the address of an overloaded function to a pointer, what is it that determines which version of the function is used?

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. Create a function called **order()** that takes two integer reference parameters. If the first argument is greater than the second argument, reverse the two arguments. Otherwise, take no action. That is order the two arguments used to call **order()** so that, upon return, the first argument will be less than the second. For example, given

   ```
   int x=1, y=0;
   order(x, y);
   ```

   following the call, **x** will be 0 and **y** will be 1.

2. Why are the following two overloaded functions inherently ambiguous?

   ```
   int f(int a);
   int f(int &a);
   ```

3. Explain why using a default argument is related to function overloading.

4. Given the following partial class, add the necessary constructor functions so that both declarations within **main()** are valid. (Hint: You need to overload **samp()** twice.)

   ```
   class samp
   {
       int a;
   public:
       // add constructor functions
       int get_a() { return a; }
   };

   int main()
   {
       samp ob(88); // init ob's a to 88
       samp obarray[10]; // non-initialized 10-element array

       // ...
   }
   ```

5. Briefly explain why copy constructors are needed.

This Page
(Not)
Intentionally Left Blank.